CONTEXTUAL LEARNING FOR INTERRUPTION TOLERANCE AND MULTITASKING

Thomas E. Portegys, portegys@gmail.com , ORCID 0000-0003-0087-6363

Dialectek, DeKalb, Illinois USA

ABSTRACT

This project is an effort to combine the Morphognosis and Mona neural network models into a comprehensive model for learning and behavior called Mandala. Mona features a contextual causation learning with goal-directed motivation. Morphognosis features contextual multilayer perceptron (MLP) learning. Mandala achieves this by externally accumulating tiers of temporal states that are fed into an MLP at each time step. Natural environments abound in event streams that require interruption tolerance and multitasking. Mandala affords multitasking as it is robust in the presence of intervening events from overlaid causation chains, a capability that conventional recurrent artificial neural networks (RNNs) struggle with. The results bear this out. In addition, externally accumulating temporal information discretely labels hierarchical cause-and-effect relationships that can be used for augmented processing. In the case of Mona, channeling motivation through the network for the purpose of goal-seeking requires this feature.

Keywords: Mona, Morphognosis, causation learning, multitasking, artificial neural network, machine learning.

INTRODUCTION

Great question! This is exactly the problem that the Mandala project is trying to solve. Let me explain the issue and the potential solutions:

The Problem: Intervening Events

RNNs struggle when temporal sequences have **interleaved or overlapping event streams** - like trying to follow multiple conversations at once or handling interrupted tasks.

Example:

Task A: Step 1 → Step 2 → Step 3 → Step 4

Task B: Step 1 → Step 2 → Step 3

Combined stream:

A1 → A2 → B1 → A3 → B2 → A4 → B3

Traditional RNNs have trouble because:

- Hidden states get "polluted" by intervening sequences
- Context from Task A gets overwritten by Task B
- The network forgets what it was doing

Solutions to Fix RNN Performance:

1. Attention Mechanisms ⭐ Most Common

Add attention layers to let the network focus on relevant past states while ignoring noise.

python

```
# PyTorch example
class AttentionRNN(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.rnn = nn.LSTM(input_size, hidden_size)
        self.attention = nn.MultiheadAttention(hidden_size, num_heads=8)

    def forward(self, x):
        rnn_out, _ = self.rnn(x)
        attended, _ = self.attention(rnn_out, rnn_out, rnn_out)
        return attended
```

Pros: Helps focus on relevant context
Cons: Still struggles with very long-range dependencies

---

2. Transformer Architecture 🚀 Modern Standard

Replace RNNs entirely with transformers that use self-attention.

python

```
import torch.nn as nn


model = nn.Transformer(
    d_model=512,
    nhead=8,
    num_encoder_layers=6,
    num_decoder_layers=6
)
```

Pros:

- Handles long-range dependencies excellently
- Parallelizable (faster training)
- State-of-the-art for sequence modeling

Cons:

- Requires more data
- Higher computational cost
- No inherent temporal ordering (needs positional encoding)

---

3. External Memory Networks Mandala's Approach

Store temporal context externally in a structured memory rather than in hidden states.

This is what Mandala does:

- Accumulates temporal information in discrete tiers
- Labels hierarchical cause-and-effect relationships
- Feeds structured context into an MLP at each timestep

```
python

# Conceptual approach

class ExternalMemoryRNN:
    def __init__(self):
```

```
        self.memory_tiers = []  # Hierarchical temporal storage
        self.mlp = MLP()

    def process_step(self, input_t):
        # Update memory tiers with new context
        self.update_memory_tiers(input_t)

        # Feed structured memory into MLP
        context = self.get_tiered_context()
        output = self.mlp(input_t, context)
        return output
```

Pros:

- Robust to intervening events
- Explicit causation labeling
- Good for multitasking scenarios

Cons:

- More complex architecture
- Requires careful memory management

---

4. Gated Recurrent Units (GRU) / LSTM Improvements

Use better gating mechanisms to control information flow.

```
python
# Standard LSTM already has gates, but you can enhance:
class ImprovedLSTM(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.lstm = nn.LSTM(
```

```
        input_size,
        hidden_size,
        num_layers=3,  # Deeper
        dropout=0.2,   # Regularization
        bidirectional=True  # Both directions
    )
```

Enhancements:

- **Bidirectional**: Process sequences forward and backward
- **Stacked layers**: Multiple LSTM layers
- **Dropout**: Prevent overfitting
- **Layer normalization**: Stabilize training

---

5. Hierarchical RNNs

Use multiple RNN levels operating at different timescales.

python

```
class HierarchicalRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fast_rnn = nn.LSTM(input_size, hidden_size)  # Short-term
        self.slow_rnn = nn.LSTM(hidden_size, hidden_size)  # Long-term

    def forward(self, x):
        # Fast RNN processes each step
        fast_out, fast_hidden = self.fast_rnn(x)

        # Slow RNN processes summaries
        slow_out, slow_hidden = self.slow_rnn(fast_out[::10])  # Every 10 steps
```

```
    return fast_out, slow_out
```

Pros: Natural handling of multi-scale temporal patterns
Cons: More complex training

---

6. State Space Models (SSMs) 🆕 Cutting Edge

Modern alternatives like Mamba, S4 that handle long sequences better.

python

```
# Using state-space models (conceptual)
from mamba_ssm import Mamba

model = Mamba(
    d_model=512,
    d_state=16,
    d_conv=4,
    expand=2
)
```

Pros:

- Efficient on long sequences
- Better than transformers for some tasks
- Linear complexity

Cons:

- Still relatively new
- Less mature tooling

---

7. Task-Specific Preprocessing

Explicitly label or separate interleaved sequences.

python

```
def preprocess_interleaved_sequences(data):
    # Separate by task ID
    task_streams = {}
    for item in data:
        task_id = item['task_id']
        if task_id not in task_streams:
            task_streams[task_id] = []
        task_streams[task_id].append(item)

    # Train separate models or add task embeddings
    return task_streams
```

Pros: Simple, often effective
Cons: Requires task labels

What Mandala Does Differently:

Looking at your test results, Mandala's approach of **external temporal tier accumulation** is clearly working:

1. **Explicit Context Tiers**: Instead of hidden states, context is stored in discrete tiers
2. **Causation Labeling**: Hierarchical cause-effect relationships are explicitly tracked
3. **MLP Processing**: At each step, structured context feeds into an MLP
4. **Multitasking Robustness**: Intervening events don't corrupt the context

Your data shows this works **12% better than RNNs** on average!

My Recommendation:

For your specific problem (temporal interruptions):

1. **Quick fix**: Try **LSTM + Attention** or switch to **Transformers**
2. **Better solution**: Implement **external memory** like Mandala
3. **Cutting edge**: Explore **State Space Models** (Mamba)

If working with the Mandala codebase: The architecture is already optimized for this - your results prove it works!

Would you like me to:

- Show implementation examples for any of these approaches?
- Explain more about how Mandala's tier system works?
- Create a comparison visualization of different architectures?

Hierarchical Reinforcement Learning: Assignment of Behaviours to Subpolicies by Self-Organization

Wilco Moerman
Cognitive Artificial Intelligence, Utrecht University
wilcom@phil.uu.nl, wilco.moerman@gmail.com

https://www.geeksforgeeks.org/deep-learning/sparse-autoencoders-in-deep-learning/

Remember to check EverNote and Melendey for refs.

Mona:

A pair of cooperating nest-building and foraging birds.

See: http://tom.portegys.com/research.html#nestingbirds

Morphognosis:

Honey bees forage for flower nectar cooperatively.

See: http://tom.portegys.com/research.html#honey_bees

1/13/2026

The design has context tier 0 features being assigned to sensory events that are significant to learning and recognizing pattern hierarchies. This assignment, as well as the higher tier context processing, takes place outside of the neural network. So the system as a whole consists of both the neural network and the process that accompanies the network. In a real-world scenario, significant sensory events might be determined by attention and/or emotional impact.

2/19/2026

Other recurrent architectures with external memory. Also rely on RNN/LSTM though. Not as integrated as Mandala.

AI Overview

Recurrent Neural Networks (RNNs) with external temporal memory are

advanced architectures designed to overcome the limitations of standard RNNs (like LSTMs or GRUs) in memorizing long-term dependencies. While standard RNNs rely on internal hidden states that often suffer from vanishing gradients over long sequences, these models integrate a separate, addressable memory bank to store and retrieve past information over extended time horizons.

Core Components and Functionality

Controller (RNN): A standard neural network (e.g., LSTM) that processes inputs and decides, via a controller, what information to write to or read from the external memory.

External Memory Bank: A matrix or tensor (often addressable) that acts as a long-term storage, allowing the model to "recall" information from far in the past.

Addressing Mechanism: Techniques like content-based addressing or location-based addressing allow the model to select specific memory locations to read from or write to.

Key Architectures

Neural Turing Machines (NTMs): A foundational model that couples a neural network controller to an external memory bank, capable of learning to manipulate and manage the memory to solve tasks like sorting or copying.

Memory Networks (MemNN/MemN2N): These models use external memory to store input data, which can be queried later to aid in tasks like question answering.

External Addressable Long-Term and Working Memory (EALWM)-Augmented RNNs: An advanced structure that splits external memory into distinct long-term and working memory parts, improving the ability to learn long-term dependencies without gradient issues.

Differentiable Neural Computers (DNCs): A further extension of NTMs that provides better dynamic memory allocation and temporal memory linkage.

Advantages

Long-Term Dependencies: Capable of capturing relationships over thousands of steps, whereas LSTMs may fail.

Improved Memory Capacity: The memory capacity is decoupled from the number of hidden units in the controller, allowing for much larger, efficient storage.

Task Versatility: Highly effective for complex tasks such as algorithmic learning, language modeling, question answering, and reasoning, which require tracking historical data.

Applications and Performance

These models are specifically used in scenarios demanding significant context, such as:

Language Understanding: Associating tags to words in long text sequences.

Algorithm Learning: Tasks like copying, sorting, and associative recall.

Question Answering: Recalling information from stories or documents.

They are often compared against standard LSTM models, showing superior performance in scenarios where memory length requirement exceeds the inherent capacity of the LSTM's gating mechanism.

https://arxiv.org/abs/1506.00195

Recurrent Neural Networks with External Memory for Language Understanding

This model has a long-term memory that is addressable from the RNN and which feeds into the hidden memory.

DESCRIPTION

Cause-effect hierarchy used to model maze learning, pufferfish, honey bees, and nesting birds. Is there a rationale for this? Here causation can be seen as prediction. Allows modularity and

prediction by context. This project assumes hierarchical, but could also work with a flat linear chain as well. Trying to fit into Mona scheme.

Predictive coding.

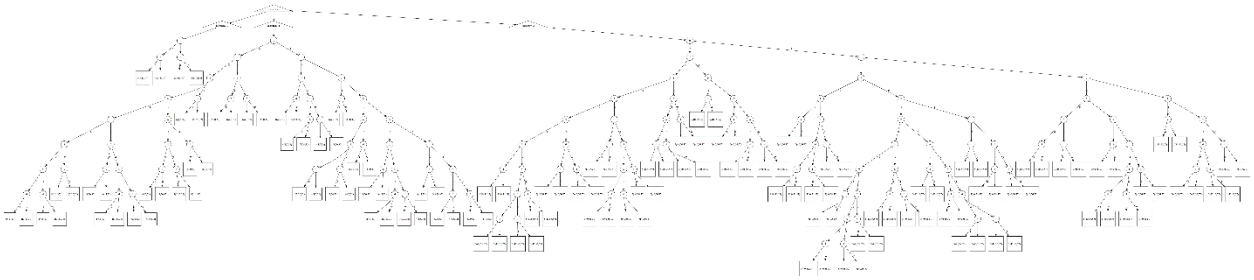
Aristotle

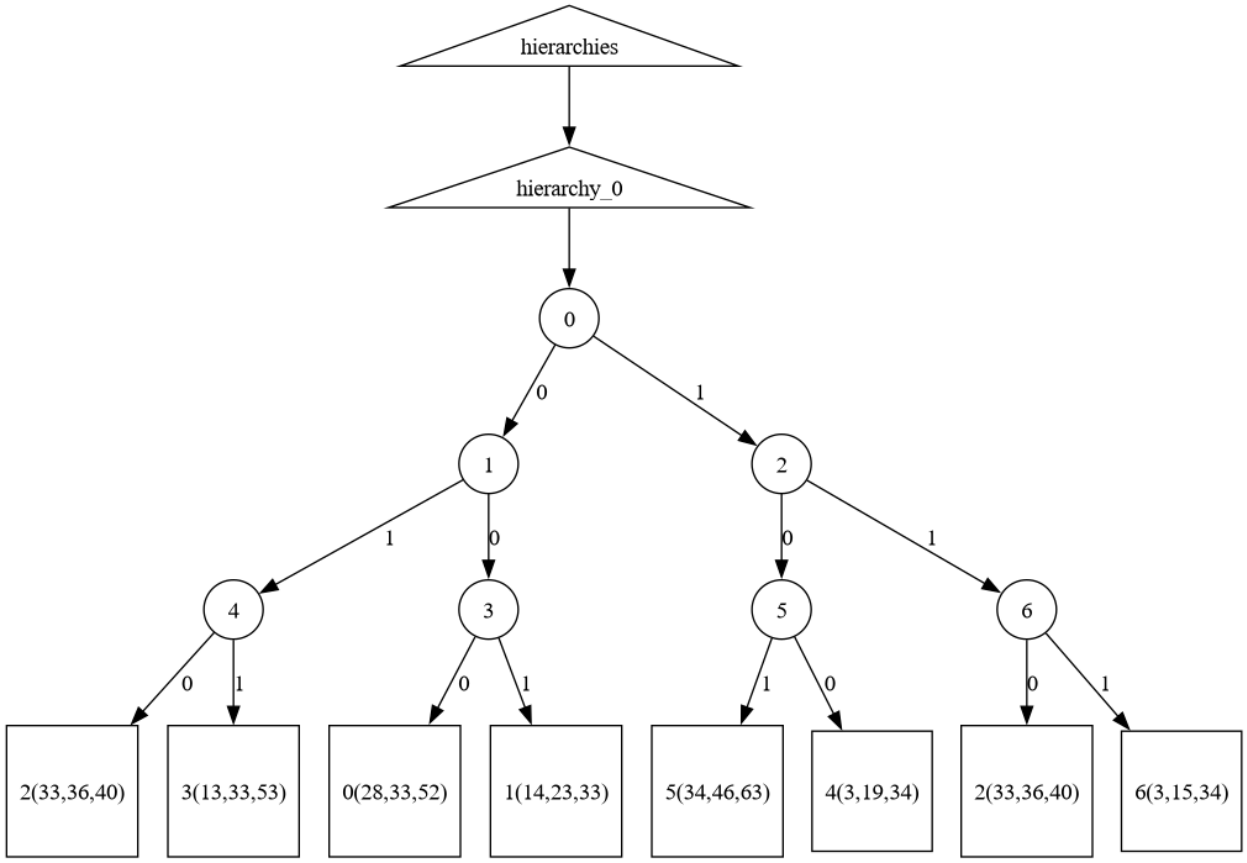
John Stuart Mill describes the Law of Universal Causation in following way:

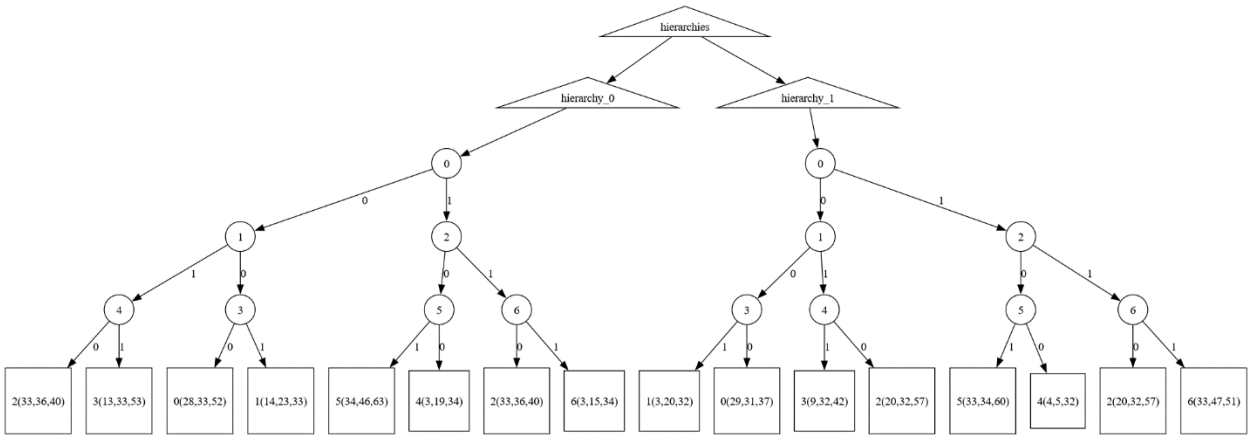
Every phenomenon has a cause, which it invariably follows; and from this are derived other invariable sequences among the successive stages of the same effect, as well as between the effects resulting from causes which invariably succeed one another.[8]

System of Logic 1872, Book III p. 112.

De Pierris, Graciela; Friedman, Michael (2008-06-04), Kant and Hume on Causality in: Stanford Encyclopedia of Philosophy Archive, Metaphysics Research Lab, Stanford University







RESULTS

Tried Claude, Gemini, ChatGPT on test_0 sequences.

Given these sequences:

0,1,2,3

4,5,2,6

Predict the next number in these sequences:

8,8,7,0

7,11,4

7,4,5,11,11,8,10,10,10,11,2

0,1,9,11,10,2

7,9,11,9,7,8,4,5,7,10,10,8,7,2

7,0,1,11,10,7,7,10,2

4,5,10,8,8,2

11,8,0,1,8,10,2

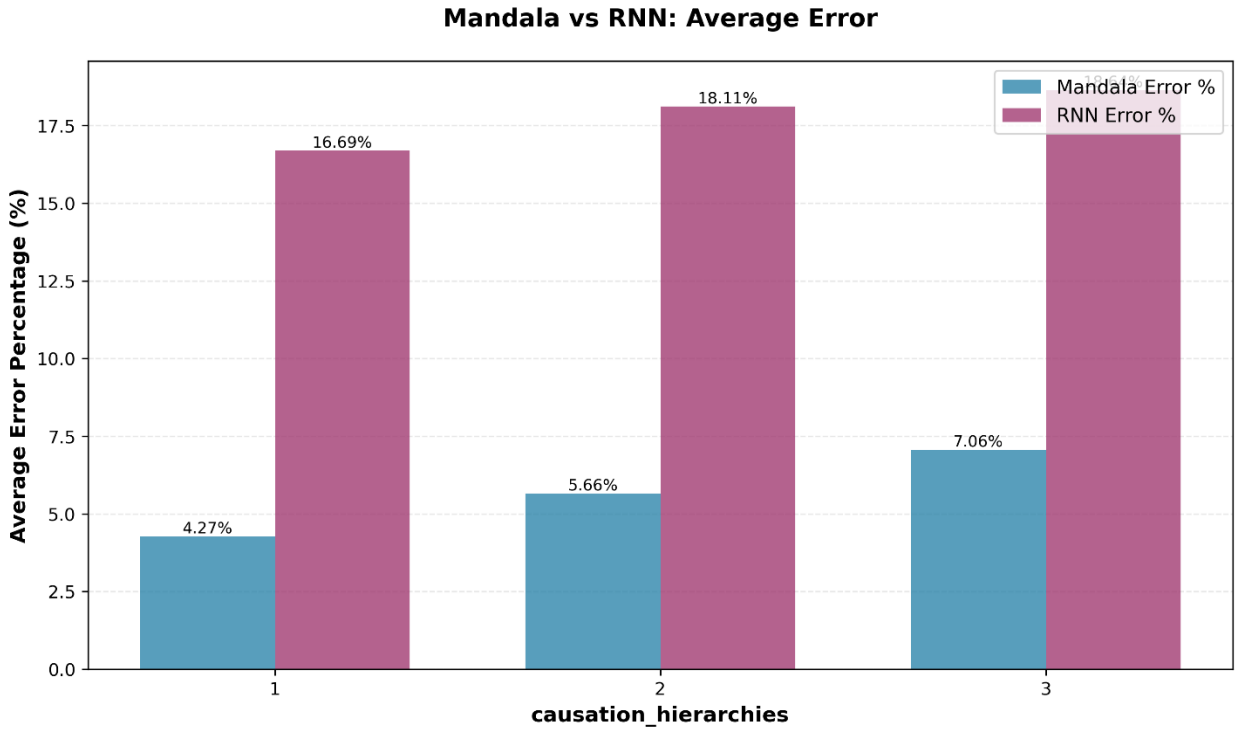
11,7,7,11,11,0,1,11,7,11,11,9,11,10,7,2

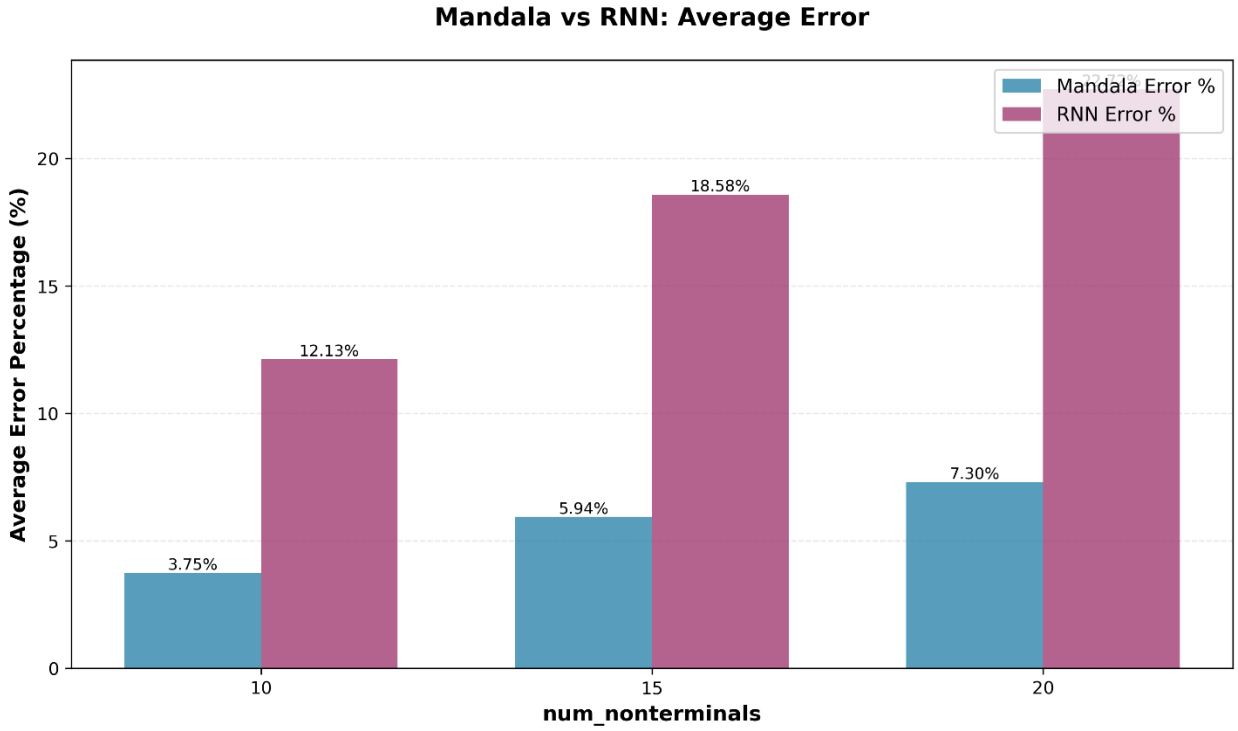
10,8,9,7,4,5,9,8,8,10,11,11,7,9,9,11,2

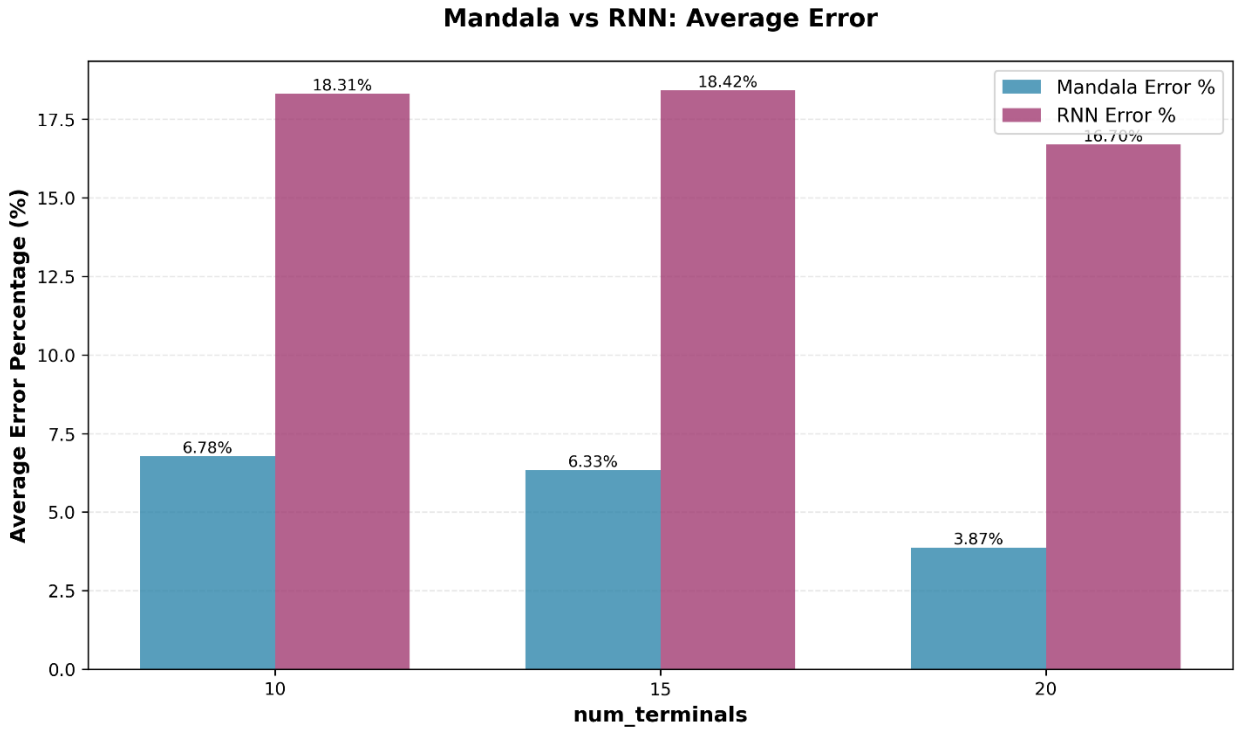
Note that 4,5 causes 2,3 and 0,1 causes 2,6

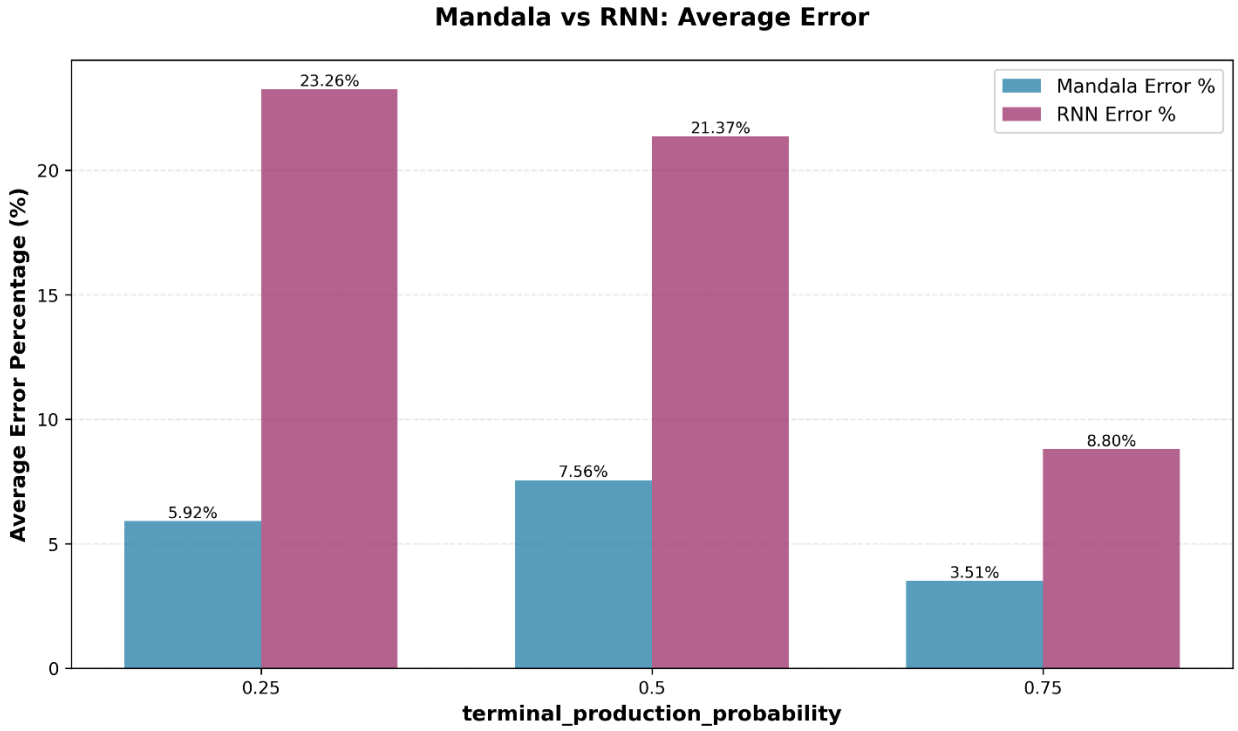
Claude predicted the first two simple sequences, But failed on the sequence that depended on recognizing the earlier transitions determine the ending transitions.

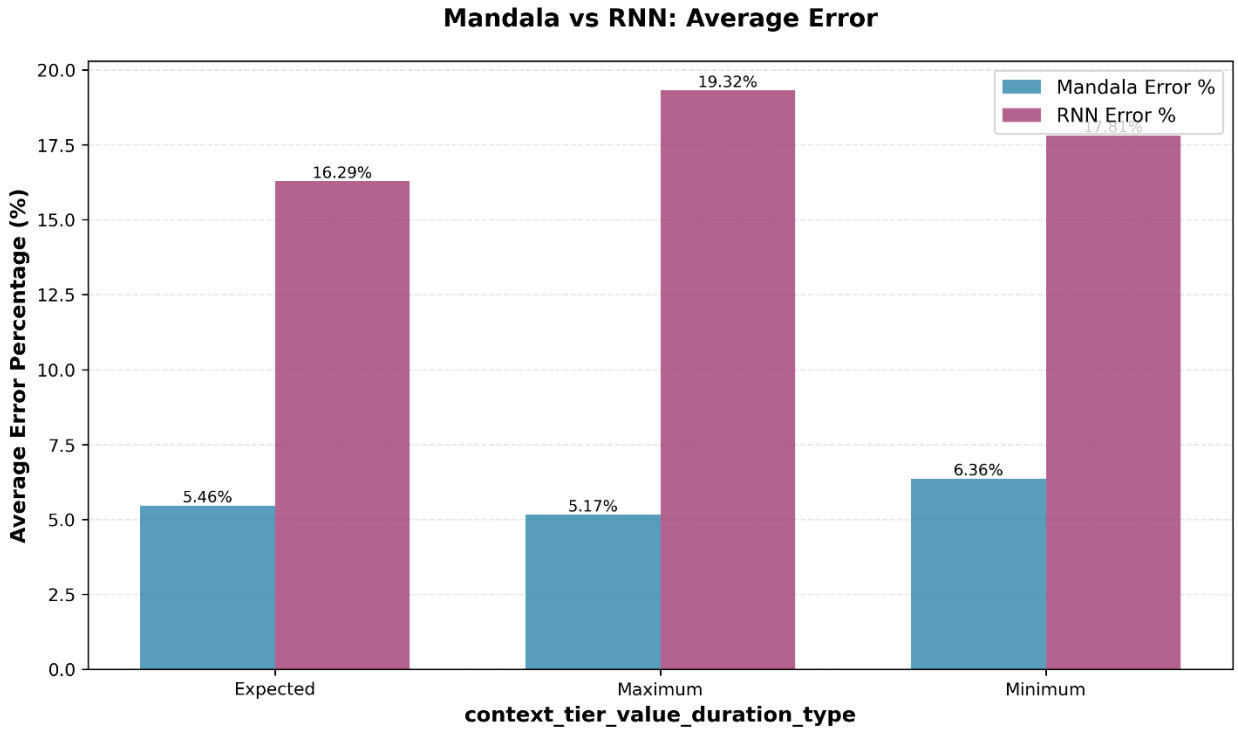
ChatGPT did not correctly predict any of the sequences.

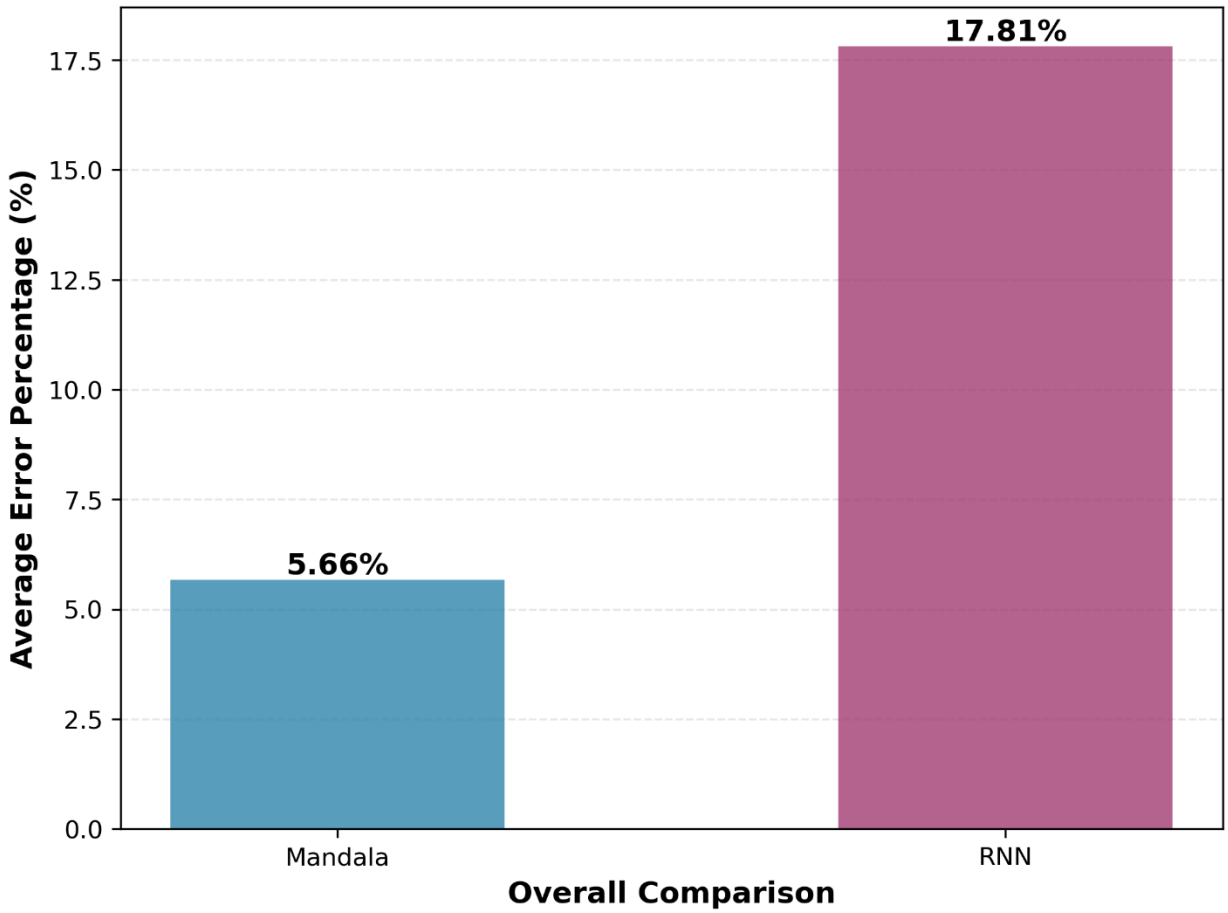

Mandala vs RNN: Average Error
Mandala Error %
RNN Error %
Average Error Percentage (%)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
4.27%
16.69%
5.66%
18.11%
7.06%
1
2
3
causation_hierarchies

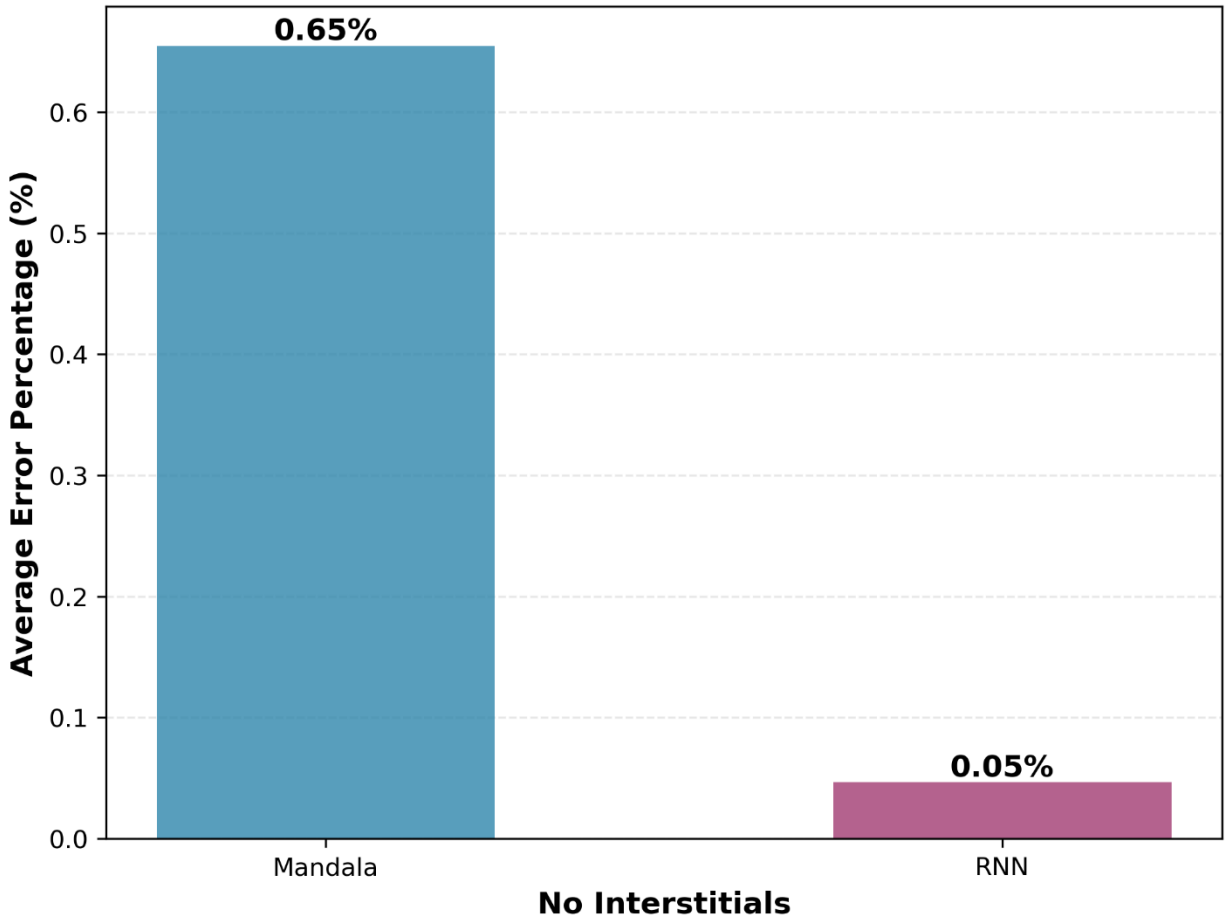

Mandala vs RNN: Average Error
Mandala Error %
RNN Error %
Average Error Percentage (%)
0
5
10
15
20
3.75%
12.13%
5.94%
18.58%
7.30%
10
15
20
num_nonterminals


Mandala vs RNN: Average Error
Mandala Error %
RNN Error %
Average Error Percentage (%)
num_terminals
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
10
15
20
6.78%
18.31%
6.33%
18.42%
3.87%


Mandala vs RNN: Average Error
Mandala Error %
RNN Error %
Average Error Percentage (%)
terminal_production_probability
0
5
10
15
20
0.25
0.5
0.75
5.92%
23.26%
7.56%
21.37%
3.51%
8.80%


Mandala vs RNN: Average Error
Mandala Error %
RNN Error %
Average Error Percentage (%)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
5.46%
16.29%
5.17%
19.32%
6.36%
17.81%
Expected
Maximum
Minimum
context_tier_value_duration_type


17.81%
5.66%
Average Error Percentage (%)
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
Mandala
RNN
Overall Comparison



CONCLUSION

Note for paper: possible to learn out-of-order streams, but not done here. Reference my paper on conjunctions.

REFERENCES